mBlox XML Interface Technical Manual

Provision of mBlox Services is dependent upon compliance with the specifications set forth herein. The information in this document is subject to change without notice. Although mBlox has taken reasonable steps to ensure the accuracy and completeness of this document, it shall not be liable for any losses whatsoever, whether direct or indirect, including without limitation any loss of profit, loss of use, or loss of data, as a result of any errors or omissions contained herein. The information or statements in this document concerning the specification or performance of mBlox software or hardware systems shall not constitute any binding promise or warranty.

Table of Contents:

1. Preface 4
1.1. Where to get help 4
2. Change History 5
2.1. Current document 5
2.2. Version History 5
3. Glossary 7
4. Introduction 8
4.1. XML interface feature set 8
4.2. Minimum requirements for clients to use the interface 8
4.3. Product and geography dependence 8
4.4. Terminology 9
4.5. Contacting mBlox Client Support 9
5. XML Interface Overview 11
6. Account Setup 14
6.1. Account-specific settings 14
6.2. Account specific information provided to the Client 15
6.3. Account specific information the Client provides to mBlox 15
6.4. XML over HTTPS 16
7. Connection Management 17
7.1. Server and network considerations 17
7.2. Failover and load sharing 19
7.3. Retry mechanism 21
8. Design Recommendations for Client's Application 23
8.1. Response to outbound actions 23
8.2. Multi-threading/parallel processing 23
8.3. Log files 24
8.4. Validation of XML against DTDs (document type definitions) 24
9. Structure and Content of Actions 25
9.1. XML document specification 25
9.2. Version control 25

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

9.3. XML transport over HTTP 26
9.4. The Notification Request 26
9.5. The Notification Request Result 35
9.6. The Notification Service 44
9.7. The Response Service 51
10. Message Body Character Sets and Encoding 58
10.1. Illegal characters in XML 58
10.2. MT message characters sets and encoding 59
10.3. MO message character sets and encoding 61
11. Appendix – Example XML Documents for Notification Request 62
11.1. Plain text SMS 62
11.2. Ringtone delivered as a binary format SMS 63
11.3. Operator logo delivered as a binary format SMS 64
11.4. WAP push SI delivered as a binary format SMS 65
12. Appendix A: Reject Codes and Reasons 66
13. Notes 68

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

1. Preface

This document is designed to take the Client through the XML Interface in detail. If it does not answer the Clients question please contact mBlox customer support, (see '**Where to get help**' below).

For definitions of some of the commonly used terms, please refer to **3. Glossary**.

1.1. Where to get help

For any further information or support please visit our website at **www.mblox.com** or contact one of our offices:

United Kingdom:	**Address:**	Olympic Office Centre, 4th Floor, 8 Fulton Road, Wembley HA9 0NU
	Tel:	+44(0)20 8432 1260
	Fax:	+44(0)20 8432 1290
USA:	**Address:**	455 West Maude Avenue, Sunnyvale, CA 94085
	Tel:	+1-408-617-3700
	Fax:	+1-408-617-3799
Sweden:	**Address:**	Tegnergaten 3, 1tr, 111 40 Stockholm
	Tel:	+46(0)853 480780
	Fax:	+46(0)853 480789
France:	**Address:**	34 Blvd Haussman, 75009 Paris
	Tel:	+33 1 72 71 25 55
	Fax:	+33 1 72 71 25 99
Spain:	**Address:**	Zurbano, 41, 1º planta, 28010 Madrid
	Tel:	+34 91 310 98 28
	Fax:	+34 91 591 43 51
E-mail:	**support@mblox.com**	

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

2. Change History

2.1. Current document

Document Title: mBlox XML Interface Technical Manual
Author: Jason French
Classification: Confidential
Version: 3.3
Date of Issue: 11th July 2011

2.2. Version History

Version	Date	Comments
1.0	10th February 2005	Update to XML Sender System Interface and XML Examples with Comments and Hints
1.1	11th May 2005	Corrected character support. Minor corrections. Clarified HTTP support. Updated DTDs for Notification Request and Notification Request Result.
1.2	13th October 2005	Update to Unicode encoding, format of <SubscriptionName>, DTDs for Notification Request, Notification Request Result and Response Service., example format of Notification Request and Response Service. Added <Tags>, <Tag>, Name, <ServiceDesc>, <ContentType> and <ServiceId>.
1.3	18th January 2006	Correction to <SenderID> character support (section 8.4.4).
1.4	26th June 2006	Multiple URL support
2.0	13th November 2006	Updating of document presentation and format
2.1	15th February 2007	Update to format
2.2	15th August 2007	Added tag and tags to Structure and content actions
2.3	8th September 2008	Update on HTTPS
2.4	3rd November 2008	Update to version 3.5 of the XML schema and cleaned up the formatting of the document
2.5	8th November 2008	Updated the Section 5, last sentence and Section 9.6.
2.6	2nd March 2009	Correction to link in 7.3.1

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

Version	Date	Comments
2.7	2nd April 2009	Correction to Tariff field format in Response Service
2.8	19th November 2009	Update to 9.6 The Notification Service and 9.5.1. Immediate Return. Addition of Appendix A
2.9	14th January 2010	Update to 7.3.2 Retry mechanism – outbound actions
3.0	28th January 2010	Correct to sample XML on p28 and p47
3.1	25th October 2010	Minor corrections.
3.2	4th November 2010	Changes to the connection type and addition of username in the notification request.
3.3	11th July 2011	Standardizing supportable format types

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

3. Glossary

Below is a list of the terms with their corresponding definitions used within this document.

- **2n redundancy**: the duplication of elements within a system to provide high reliability
- **API**: Application Program Interface
- **DNS**: Domain Name Service
- **DTD**: Document Type Definition
- **HTTP**: Hyper Text Transport Protocol
- **HTTP POST**: A method used to send data to a server via an HTTP request
- **IP**: Internet Protocol
- **IP address**: An identifier for a device on an IP network
- **MO**: Mobile-Originated
- **MSISDN**: Mobile Station Integrated Services Digital Network number (as defined in ITU-T E.164)
- **MT**: Mobile-Terminated
- **PSMS**: Premium Short Message Service
- **PTT**: Premium Transaction Tracking – An mBlox product which provides a detailed and uniform set of error codes in relation to operator delivery receipts
- **SMS**: Short Message Service
- **SMSC**: Short Message Service Center
- **URL**: Uniform Resource Locator
- **URL-encoding**: An encoding method in which certain restricted characters are replaced by an escape sequences
- **XML**: eXtensible Markup Language

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

4. Introduction

This manual provides the technical guidance required to integrate to the mBlox platform using a proprietary, XML-based interface. This interface is designed to provide Clients with a straightforward way to create a connection to mBlox. It is easy to integrate into applications and requires skills and knowledge that are familiar to web developers.

4.1. XML interface feature set

- Communication via XML over HTTP transactions
- Provides access to full mBlox product set of premium and non-premium messaging services
- Duplication of mBlox HTTP servers provides 2n redundancy
- Robust mechanisms for failover and retry management
- Supports mobile terminated (MT) messaging, with a single message to an individual subscriber submitted per XML/HTTP transaction
- Supports confirmation of acceptance of MT messages by mBlox platform
- Supports delivery receipts (status information related to MT messages returned from operator/carrier network)
- Supports mobile originated (MO) messaging, with a single message from an individual subscriber returned per XML/HTTP transaction

4.2. Minimum requirements for clients to use the interface

- Hosting of an HTTP server (if support for MO messaging and delivery receipts is required)
- Programming knowledge and experience
- Working knowledge of HTTP transactions
- Understanding of XML

4.3. Product and geography dependence

mBlox operates two platforms, one based in the USA, the other in Europe. There are minor differences in the way in which the Client uses the XML interface to connect to each one. This manual is applicable to both geographies and where such differences occur, they are clearly noted.

Clients connect to mBlox to take advantage of a range of messaging products. This manual covers the use of the XML interface across the whole range of mBlox products. Where there are product-specific items in this manual, they are clearly noted. The product(s) which the Client uses will determine whether these are applicable.

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

Where relevant, the Client is directed to the appropriate geography-specific or product-specific documentation.

4.4. Terminology

The following terms have specific definitions in this manual.

- **Client** – To avoid confusion, throughout this manual "Client" (uppercase "C") is used to denote a customer of mBlox. The use of "client" in the technical, networking sense (e.g. client-server model) is avoided
- **Message** – The term "message" is used specifically to refer to the Short Message Service (SMS)
- **Action** – The term "action" is used specifically to refer to either a request or a response passed across the XML interface i.e. it is either half of a transaction. It is used to avoid confusion with the term "message", defined above
- **Inbound/outbound** – the term "inbound" refers to data flow in a direction towards the mBlox platform, while the term "outbound" refers to data flow in a direction away from the mBlox platform
- **Operator/carrier** – "carrier" and "operator" are equivalent terms, both referring to telecoms companies which operate the core mobile network infrastructure and provide services to the subscriber. In this manual the term "operator" is used

4.5. Contacting mBlox Client Support

This manual is intended to provide sufficient guidance for a software developer to design and implement an application or API which can communicate with the mBlox platform using the XML interface. Please read it thoroughly before beginning any design work.

If the developer encounters questions or problems in the course of designing and testing their application, which cannot be resolved by reference to this manual, please refer to the trouble shooting guide for the XML interface or contact mBlox Client Support. Contact details are provided when the Client sets up an account with mBlox. Note that end-to-end testing (Client application to subscriber handset) should be performed by the Client when they set up their service. mBlox can only assist with debugging problems related to the interface between the Client and mBlox platform.
In order to help mBlox provide speedy and accurate resolution of technical issues, the Client should provide as much detail as possible. This includes the exact nature of the issue, the account details, and, if possible, relevant extracts from log files (to include timestamps, originator/destination numbers, ProfileIDs, error codes etc.).

Please note that it is not possible for mBlox to provide application design or consultancy services.

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

This manual provides guidance on how an application should be integrated with mBlox's XML interface and a few design recommendations in this regard. While every effort will be made to support the Client's testing of the interface and troubleshooting, mBlox is not able to validate the Client's software design or code.

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

5. XML Interface Overview

The XML interface is designed to provide a straightforward means of connecting a Client's messaging application to the mBlox platform. Mobile terminated (MT) messages pass from the Client's application to mBlox, for submission to the operator network, while delivery receipts and mobile originated (MO) messages from the operator network are passed back to the Client's application.

At the network level, the interface is based on HTTP transactions over the public Internet, between a server on the mBlox side and a server operated by the Client. Data to be passed over the interface is formatted into an XML document, and placed in the body of an HTTP request or response.

Figure 1, overleaf, illustrates the three types of HTTP transaction which occur across the interface. In this manual, either half of a transaction (either request or response) is referred to as an "action". Thus, six types of action (each half of three transactions) occur across the interface, and are outlined in **table 1**.

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

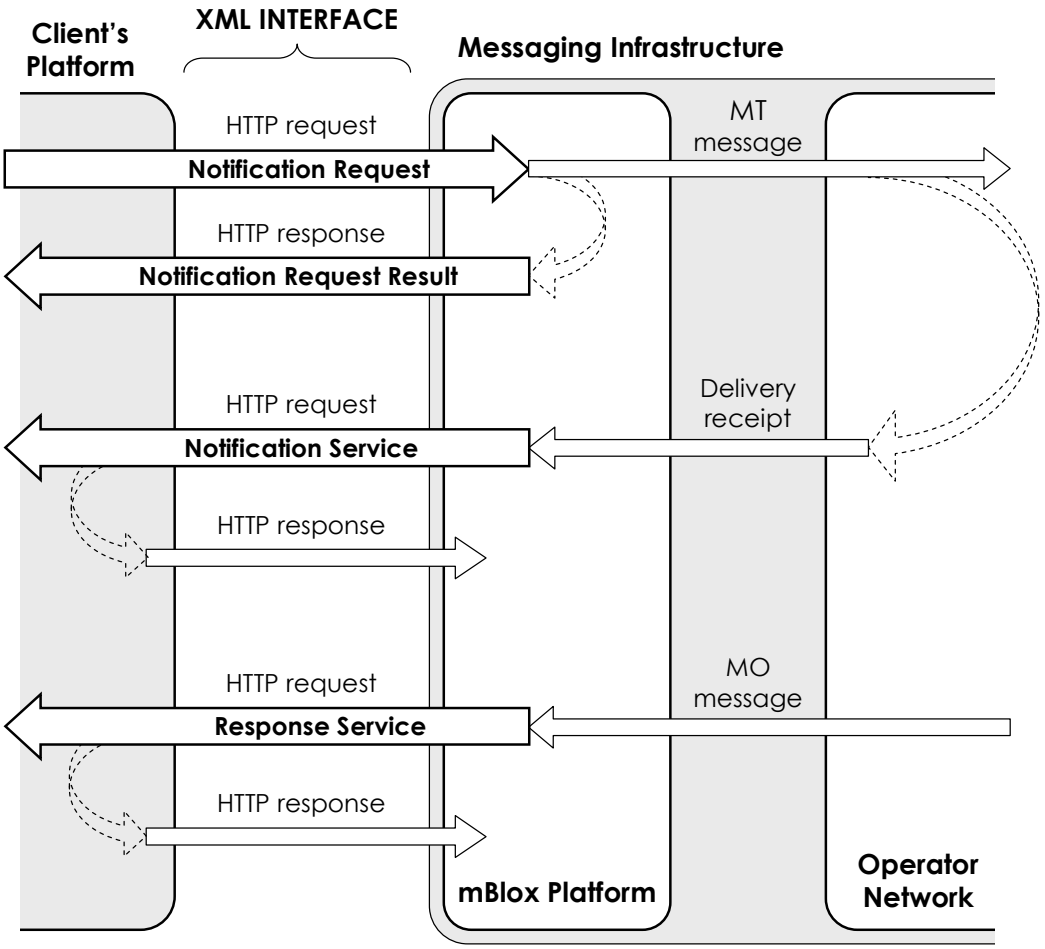

Figure 1 – Transactions across the XML interface

Originator	HTTP Request (using POST method)	HTTP Response
Client	Body contains XML document Notification Request.	Body contains XML document Notification Request Result.
mBlox	Body contains XML document Notification Service.	HTTP header only. Body content ignored.
mBlox	Body contains XML document Response Service.	HTTP header only. Body content ignored.

Table 1 - The six actions (three transactions) which occur across the interface

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

Note that all of the HTTP requests use the POST method.

Four of the types of action carry an XML document and are given specific names.

- **Notification Request** – This is used to submit a single MT message intended for an individual subscriber from the Client application to the mBlox platform. The mBlox platform returns an HTTP response containing a Notification Request Result
- **Notification Request Result** – This is returned to the Client application to acknowledge the receipt of a Notification Request. It indicates the success/failure of the Notification Request to be processed by the mBlox platform. It is returned in the HTTP response to the Notification Request
- **Notification Service** – This is used to submit a single delivery receipt (delivery status information related to a specific MT message originated within the operator network) from the mBlox platform to the Client's application. The Client's server must provide an HTTP response to the request in which the Notification Service is supplied
- **Response Service** – This is used to submit a single MO message from the mBlox platform to the Client's application. The Client's server must provide an HTTP response to the request in which the Response Service is supplied

Response Services are only provided on accounts which access an MO messaging product.

The detailed structure and content of each of these four actions is described in 9. Structure and Content of Actions

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

6. Account Setup

6.1. Account-specific settings

The Client is provided with one or more accounts, to allow access to the mBlox platform. mBlox offers both premium SMS (PSMS) and non-premium SMS products. Premium and non-premium services cannot be accessed on the same account. Clients using both types of product will be provisioned with two separate accounts.

When an account is provisioned, a number of options associated with it must be set up by mBlox. Some option settings are determined by the products which are being activated on the account. Others may be set according to the Client's preferences as to how they will use the XML interface.

6.1.1. Product-dependent account settings

The following account options are determined according to the products activated on the account.

- **Mobile originated messaging** – MO messaging (via the Response Service) is only provided on non-premium accounts for which an MO product has been activated, but is provided as standard on premium SMS accounts. Information throughout this manual in relation to MO messaging and the Response Service is only applicable in these cases
- **Premium Transaction Tracking** – In some geographies the Premium Transaction Tracking (PTT) product is available as an enhancement to PSMS. It provides detailed, uniform information about the delivery and billing status of an MT message. If activated on an account it affects the information contained in the Notification Service (**section 8.6.3**). The Client will find further information in the PTT Technical Guide for their geography

6.1.2. Client preference account settings

The following account options are set according to Client preference.

- **Acknowledgement scheme** – This may be selected as Immediate Return (non-blocking mode) or Reported Return (blocking mode) and affects the way in which Notification Request Results are generated (**9.5. The Notification Request Result**). The default setting is Reported Return. **Notification Services** – These may be enabled or disabled. If during provisioning the Client provides a URL at which to receive Notification Services (**6.3. Account specific information the Client provides to mBlox),**

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

they will be enabled by default. Information throughout this manual in relation to delivery receipts and Notification Services is only applicable if they are enabled

6.2. Account specific information provided to the Client

The following information is necessary for use of the XML interface and is among the details provided to the Client from mBlox when setting up new accounts.

- **Account User Name** – This uniquely identifies the relevant Client account when requests are made in either direction across the interface. Clients with more than one account should take particular care to use the correct User Name for the account they wish to access
- **Account Password** – This is used in conjunction with the account User Name to provide authentication when a request is made in either direction across the interface. A separate password is provided for each account that the Client uses
- **ProfileID** – The Client is provided with a unique numeric identifier (termed a ProfileID) for each MT product (both premium and non-premium) to which they have access. This parameter is included in the Notification Request (**9.4. The Notification Request**) and identifies the messaging product over which a message should be sent when it is submitted to the mBlox platform

! **Clients with more than one ProfileID should take particular care to use the correct one for the product they wish to access.**

- **mBlox URL(s)** – The Client is provided with URL(s) for the resources on the mBlox platform which will listen for Notification Requests. For the **European platform,** two different URLs are provided, while for the **US platform** there is only one due to differences in network architecture. This is explained fully in **7.1. Server and network considerations.**

6.3. Account specific information the Client provides to mBlox

The following information must be provided by the Client (as applicable) when setting up new accounts.

- **Client URL for receiving Notification Services** – In order to enable Notification Services on an account the Client must specify a URL for the resource which will listen for HTTP requests from mBlox. Note that the Client is free to specify either the same or a different URL from the one used for Response Services
- **Client URL for receiving Response Services** – In order to enable Response Services on an account, the Client must specify a URL for the resource which will listen for HTTP

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

requests from mBlox. Note that the Client is free to specify either the same, or a different URL from the one used for Notification Services

URLs may be specified in terms of either **http://<IP address>:<port number>/<path>**, or **http://<hostname>:<port number>/<path>**. If no port number is specified, port 80 will be assumed.

6.4. XML over HTTPS

Clients can send and receive XML traffic over HTTPS if additional security is required for the client connection to mBlox. The security is set up as inbound, MO and Delivery Notification traffic, and outbound, MT Traffic.

6.4.1. Inbound (MO and Delivery Notification traffic)

Clients can supply secure URLs for mBlox to post MO and Delivery Notification traffic. Client can use either the default ports HTTP, Port 80, and HTTPS, Port 443, or they can supply specific ports in the URL.

So a standard HTTP mo/status URL may look like, 'http://www.mysite.com/myreceiver', which would be the same as 'http://www.mysite.com:80/myreceiver'. For HTTPS the URL would look like, 'https://www.mysite.com/myreceiver' which would be the same as 'https://www.mysite.com:443/myreceiver'

Clients may listen on any port for their application (mBlox uses 8180, as an example) in this case the port has to be contained in the URL. For example, the new URL would then become 'http://www.mysite.com:1111/myreceiver' or 'https://secure.mysite.com:2222/myreceiver'

6.4.2. Outbound (MT traffic)

Clients can submit MT traffic to mBlox as either secure or non-secure.

- Non-secure traffic is submitted via Port **8180**, to URL: **'http://xmlX.mblox.com:8180/send'**.
- Secure traffic is submitted via port **8443**, to URL: **'https://xmlX.mblox.com:8443/send'**

mBlox client can use either mBlox self-signed certificates, or third party authenticated certificates depending on their requirements for security levels. The default will be for all clients to use mBlox self-signed certificates, if you have a requirement to use a certificate authenticated by a third party please let your Account Manager know when the service is being set up.

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

7. Connection Management

7.1. Server and network considerations

7.1.1. 2n server redundancy

European platform only

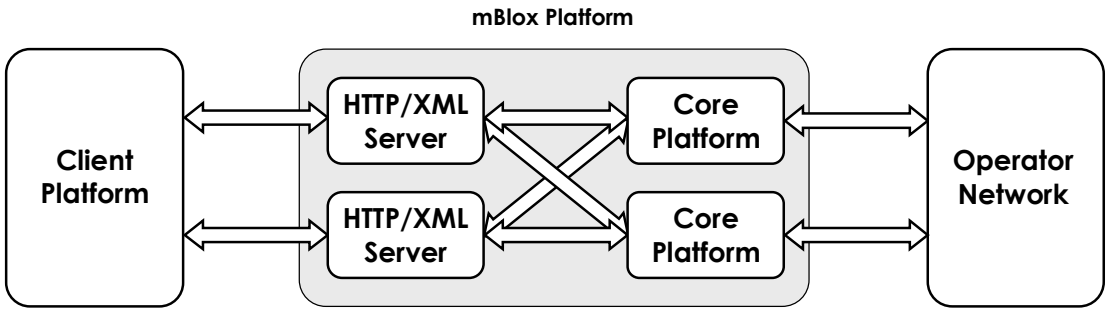

Figure 2 – mBlox HTTP server redundancy

Two separate mBlox HTTP servers handle the XML interface for each Client. They provide identical functionality and are located in separate data centers, thus forming a 2n redundant system at the network level. The Client must manage connections to both servers to receive full availability of the interface, and benefit from this resilience. Section 6.2 details how failover is implemented across the interface. Note that the core platform is also fully duplicated to provide redundancy.

US platform only

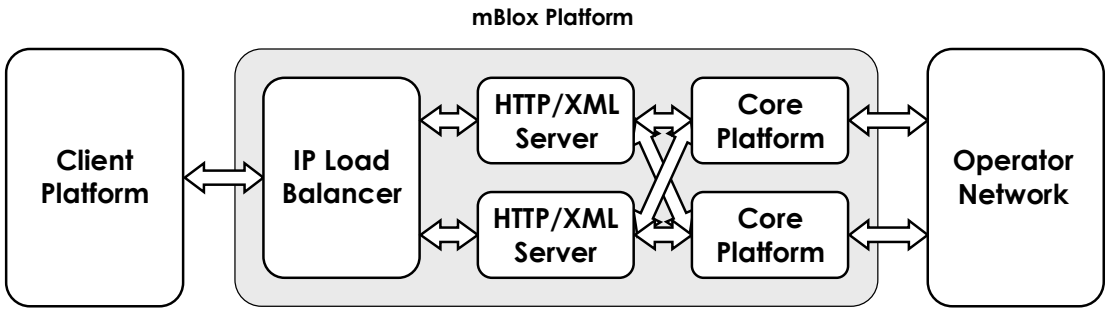

Figure 3 – mBlox HTTP server redundancy incorporating IP load balancer

Two separate mBlox HTTP servers handle the XML interface for each Client (as in the European platform), providing 2n server redundancy.

However, mBlox operates an IP load balancer in front of the servers, so that the Client only connects to a single IP address. Thus, the Client does not have to handle failover

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

management themselves. Note that the core platform is also fully duplicated to provide redundancy.

7.1.2. URLs and IP address resolution

The Client is provided with URL(s) for the listener resources on the mBlox servers in the following formats:

European platform

- http://xml<*first server number*>.mblox.com:8180/send
- http://xml<*second server number*>.mblox.com:8180/send

US platform

- http://xml.us.mblox.com:8180/send

mBlox may change the IP addresses on which these resources are hosted.

The Client should configure their server not to use cached IP addresses resolved from these DNS addresses. Otherwise, they will encounter connection problems when mBlox makes IP address changes.

If the Client is configured to receive Notification or Response Services, they must provide a URL pointing to the listener resource for each action. This may be the same resource for both types of action, or a separate one for each. mBlox performs limited lifetime caching of the IP address resolved from a Client's hostname.

Multiple URL's

Each Redhot client can be configured to have multiple usernames. Each of these usernames can be configured in Redhot to map to a unique client URL.

To send on a specific username the sender must add the tag <username></username> to the header of the notification (notificationrequest-3_5.dtd).

All delivery receipts for a specific username will be delivered to the URL associated with that specific username.

The Client must notify mBlox in advance of any changes to the URLs or associated IP addresses for their listener resources to prevent a temporary loss of outbound connections.

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

7.1.3. Firewall configuration

The Client must ensure that their firewall configuration (if applicable) does not inhibit transactions between their server and the mBlox servers. With inbound requests, this includes keeping sockets open to responses from mBlox for a sufficient amount of time. If the Client's application has initiated a transaction and is expecting a response through a socket, the firewall should not block responses to that socket until after the process which is waiting has timed out. (7.2. Failover and load sharing provides information about how the Client's application should react to a timeout.)

If the Client is configured to receive Notification or Response Services, their firewall configuration should allow outbound connections to the appropriate socket from the mBlox platform. For the European platform, this includes connections from either XML server, while for the US platform this means connections from the IP load balancer.

7.1.4. Network latency

The Client should test the ping time from their server to the mBlox platform i.e. to both XML servers for the European platform or to the IP load balancer for the US platform. If the ping time is in any case greater than 50 milliseconds, the Client should check their connection to the Internet. Excessive network latency may impact on the message throughput achievable over the interface.

7.2. Failover and load sharing

7.2.1. Failover and load sharing – European platform only

It is required that Clients on the European platform follow the advice in this section, in order to provide full availability of the connection to mBlox. mBlox operates two HTTP servers to support the XML interface, providing full 2n redundancy at the network level. The functionality provided by each server is identical and in normal operation, both are in service. However, either of them may be out of service from time to time.

mBlox only guarantees that at least one of the servers will be available at any point in time. The Client must implement connections to both of them to achieve full availability of the link.

7.2.2. Failover and load sharing – European platform – inbound actions

! **It is strongly recommended that Notification Requests, originated by the Client, are rotated between both XML servers during normal operation.**

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

Load sharing using both servers maximizes the message throughput that the Client can obtain. It also provides regular confirmation that both paths are alive and that the Client has a failure-tolerant connection.

Failover management must be handled by the Client's application when one of the mBlox servers is unable to respond successfully. It should be triggered by either of the following failures occurring in response to an HTTP request:

- The mBlox server does not respond within 60 seconds of the request
- The mBlox server responds with an HTTP error (e.g. "500 Server Error")

If this trigger occurs, the Client's application should force all further requests (including the "failed" POST which produced the trigger) to be routed to the other server for a period of 5 minutes from the failure.

It is possible that a failover trigger may occur on the second server within 5 minutes of the first failure. The Client application should respond to this second failure in exactly the same way and force all further requests to be routed to the original server for a period of 5 minutes from this new trigger. This sequence of events may occur if the servers have been brought back online very shortly one after the other.

Note that it is possible that the Client's application may receive HTTP errors back from both servers, if it attempts an unsupported request (e.g. an HTTP GET, or a POST in an unsupported character encoding). In this instance the Client should check the nature of the error in their log file, before contacting mBlox Client Support.

7.2.3. Failover and load sharing – European platform – outbound actions

! **Outbound actions may or may not be configured on a Client account, depending on product selection and/or Client preference.**

mBlox will originate requests from either of its HTTP servers during normal operation. Hence, the Client must ensure that its server is capable of receiving and responding to HTTP POSTs from both mBlox servers, e.g. firewalls are correctly configured (**7.1.3. Firewall configuration**).

In the event of one of mBlox's servers being offline, all requests will be originated from the server which is in service.

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

Note that Notification Request Results are returned in the HTTP response to a Client-originated request, and will, therefore, always come from the mBlox server to which the request was sent.

7.2.4. Failover and load sharing – US platform only

The failover and load sharing mechanism in both directions is managed entirely within the mBlox platform. The Client's platform need only handle transactions with the mBlox IP load balancer. In the event that the Client's application does not receive a response to an HTTP request within 60 seconds it should begin a retry cycle, as detailed in section 6.3.1.

7.3. Retry mechanism

7.3.1. Retry mechanism – inbound actions

The Notification Request Result indicates whether a Notification Request has been accepted by the mBlox platform. In the case of a rejection this may be due to a permanent error associated with the Notification Request (this Notification Request should not be resent), or a temporary condition within the mBlox platform (the Client's application may attempt to retry the request, without modification). A retry parameter is included in the Notification Request Result XML document to indicate which of these is correct. (See section 9.5.7 Explanation of XML nodes in the Notification Request Result)

Once a Notification Request has been placed in the retry cycle, the Client application should attempt to resend it once every 30 seconds.

No more than 5 requests should be allowed in the retry cycle at any one time. If this is exceeded, further Notification Requests should be queued rather than sent. A request should expire after 4 hours in the retry cycle.

7.3.2. Retry mechanism – outbound actions

Outbound actions may or may not be configured on a Client account, depending on product selection and/or Client preference (6.1. Account-specific settings).

The Client's server must respond to outbound actions with a simple HTTP response header (with the response status "200 OK"). No content is expected in the response body and any which is present will be ignored.

If the mBlox server does not receive a response to a POST within 60 seconds (or receives an HTTP error), it will begin retrying the request periodically (for a period of time which is product dependent). Retried MO messages with the same sequence number should be de-duped by the Client to avoid processing the MO message more than once. The

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

expected MO response to the message should be sent as this is what is required by mBlox to stop retransmitting the MO message.

The mBlox platform employs a transmission window which limits the number of requests that may be in the retry cycle at any one time. If none of the requests in this window have received a response (i.e. the window is full), all other outbound actions will be queued. In extreme cases where no responses are received the queue may fill up.

Notification or Response Services will be discarded once the queue is full or if they expire within the queue.

While the mBlox platform is retrying outbound requests the throughput of both inbound and outbound actions may be reduced.

For these reasons it is vital that the Client's application responds promptly to all outbound requests. **8.1. Response to outbound actions** provides advice on aspects of the application architecture, for robust handling of HTTP responses and minimizing the occurrence of problems.

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

8. Design Recommendations for Client's Application

This section provides a few guidelines on application architecture to ensure robust handling of the mBlox XML interface.

8.1. Response to outbound actions

If Notification or Response Services are enabled on an account it is very important that responses to outbound requests are provided promptly. If the data from these is to be read into a database the following sequence of events is recommended.

- Listener process receives request
- XML document is parsed and data items are written to a buffer
- Response is generated to complete HTTP transaction
- Independent process reads data items from buffer into database

Outbound actions may be generated at a high throughput by the mBlox platform. By isolating the database access from the response process the response time should be improved. The aim is that the application should generate the response, on average, within 100 milliseconds of the request being received by the listener. Slower response times may impact the throughput of both inbound and outbound actions.

If a valid XML document has been received and parsed the application should **always** respond to complete the HTTP transaction. The response should **not** be inhibited by unexpected data in a valid XML document, for example unexpected characters or a duplicate document. Outbound actions which do not receive a response will be retried by the mBlox server (**7.3.2. Retry mechanism – outbound actions**).

8.2. Multi-threading/parallel processing

The Client's application should create separate threads/parallel processes for dealing with each inbound and outbound action. Thus, it should be able to:

- Submit multiple Notification Requests while waiting for a response to a previous outbound action (up to the maximum window size specified later in this section)
- Listen for Notification or Response Services while already processing an outbound action
- Process inbound actions and outbound actions independently and simultaneously

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

If the application is able to parallel process in this way it should employ a transmission window for inbound actions i.e. there should be a maximum number of Notification Requests which can be submitted before at least one of them receives a response. The window size should be set to no greater than 5. Refer also to **7.3. Retry mechanism** for information on retry management.

8.3. Log files

It is strongly recommended that the Client's application is designed to store as much information as possible in log files. It is recommended that for inbound actions, Notification Requests which return errors should be logged as in **table 2.**

Error	Information Logged
HTTP Error	Timestamp, HTTP response status line, URL/IP address used for POST, firewall event log
Notification Request Result returned with error result code	Timestamp, full XML of Notification Request, full XML of Notification Request Result

Table 2 - Information logged for errors on inbound actions

If disc space allows this information should be written to the log file for all inbound actions (not just errors).

For outbound actions it is recommended that, if possible, the Client's application writes raw XML documents to a log file as they are received. This allows the application to recover from temporary problems with its back-end processes by re-reading raw data from the log file.

Having detailed log file information available greatly assists mBlox in providing quick resolution in the event of problems.

8.4. Validation of XML against DTDs (document type definitions)

The Client's application may use the DTDs provided by mBlox to validate outbound XML documents but it is not necessary to do so.

If validation is performed the Client must ensure that they update the DTD in use when mBlox performs revisions to the interface, otherwise the validation process will fail. mBlox will notify the Client in advance of such revisions, and will provide updated DTDs.

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

9. Structure and Content of Actions

This chapter provides detailed information about the structure and content of the actions which are passed across the interface, including the format of the XML documents themselves.

9.1. XML document specification

For each of the four actions which contain an XML document the following is provided.

- An example of the correct format for the XML document
- A formal document type definition (DTD) against which the XML document may be validated
- A full list of the defined nodes (elements and attributes) for the document, along with the format and interpretation of the data which they contain

The convention in this manual is for element names to be written in plain typeface enclosed in angle brackets (as their <tag> would appear), and attribute names to be written in *italics*.

XML is case sensitive, e.g. <NotificationRequest> is not the same as <notificationrequest>.

9.2. Version control

Each of the four XML document types conforms to a different DTD-validated structure. As mBlox upgrades the interface, version control is applied **independently** for each document type. Hence the Client's application should **not** expect that outbound XML documents (Notification Request Result, Notification Service, Response Service) will be of the same version number as the Notification Request which the application sends.

mBlox always sends XML documents conforming to the latest version number. It will notify the Client in advance of the release of a new XML document version and will provide an updated DTD. Each XML document declares the version number to which it conforms, within one of its attributes.

It is recommended for robustness that the Client's application ignores the value of the Version attribute in outbound documents.

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

The version for each document type specified in this manual are:

- Notification Request – version 3.5
- Notification Request Result – version 3.5
- Notification Service – version 2.3
- Response Service – version 2.3

9.3. XML transport over HTTP

All transactions over the interface are initiated with an HTTP POST.

The GET method is not supported.

The body of the POST contains a single parameter, "XMLDATA". The value of the parameter is a URL-encoded text string containing the XML document.

The XML document itself should be encoded as ISO-8859-1 (Latin-1).

Most application servers will automatically handle the generation of the appropriate HTTP header and correct use of URL-encoding when performing an HTTP POST. The following information is provided for completeness and to help troubleshoot problems that may be associated with the HTTP layer.

- The Client's HTTP server should support HTTP version 1.1 (although the Client application may POST to the mBlox server in either version 1.0 or 1.1).
- The HTTP request should include the header line:

  Content-Type: application/x-www-form-urlencoded

In the body of the HTTP request, the declaration "XMLDATA=" should **not** itself be URL-encoded (otherwise the mBlox server will interpret it incorrectly and return an error) but the value following it (the XML document, itself) should be URL-encoded.

9.4. The Notification Request

9.4.1. Batch MT messaging

A Notification Request supports the submission of a single message to an individual recipient subscriber.

Multiple messages may be submitted using an individual Notification Request per message.

9.4.2. Format of the Notification Request

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

The Notification Request XML document consists of a header providing account identification/authentication, followed by a list of the messages being submitted to the mBlox platform.

Although the format of the XML supports multiple messages in this list, the Client application must only send one message to an individual subscriber in a single Notification Request.

The message is submitted in the <Notification> element which contains various pieces of message-specific operational data and the message body.

```
<?xml version="1.0"?>
<NotificationRequest Version="3.5">
    <NotificationHeader>
   <PartnerName>UserName</PartnerName>
   <PartnerPassword>Password</PartnerPassword>
    </NotificationHeader>
<NotificationList BatchID="1">
   <Notification SequenceNumber="1" MessageType="SMS">
      <Message>This is an MT message.</Message>
      <Profile>1234</Profile>
      <SenderID Type="Alpha">The_Sender</SenderID>
      <ExpireDate>03211100</ExpireDate>
      <Operator>99999</Operator>
      <Tariff>150</Tariff>
      <Subscriber>
         <SubscriberNumber>447900000000</SubscriberNumber>
         <SessionId>SessionID</SessionId>
      </Subscriber>
      <Tags>
            <Tag Name="Number">56</Tag>
            <Tag Name="City">Paris</Tag>
      </Tags>
      <ServiceDesc>ringtone1</ServiceDesc>
      <ContentType>3</ContentType>
      <ServiceId>5678</ServiceId>
   </Notification>
    </NotificationList>
</NotificationRequest>
```

9.4.3. Formal XML DTD of the Notification Request

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

```
<!ELEMENT NotificationRequest (NotificationHeader, NotificationList)>
<!ATTLIST NotificationRequest
    Version CDATA "3.5"
>
<!ELEMENT NotificationHeader (PartnerName, PartnerPassword, Username?,
SubscriptionName?)>
<!ELEMENT PartnerName (#PCDATA)>
 <!ELEMENT PartnerPassword (#PCDATA)>
 <!ELEMENT Username (#PCDATA)>
 <!ELEMENT SubscriptionName (#PCDATA)>
 <!ELEMENT NotificationList (Notification+)>
 <!ATTLIST NotificationList    BatchID CDATA #IMPLIED
>
<!ELEMENT Notification (Message, Profile?, Udh?, SenderID?, ExpireDate?,
Operator?, Tariff?, Subscriber+, Tags?, ServiceDesc?, ContentType?, ServiceId?
)>

<!ATTLIST Notification
    SequenceNumber CDATA #REQUIRED
    MessageType (SMS | FlashSMS) #IMPLIED
    Format (Unicode | Binary | Imode) #IMPLIED
>
<!ELEMENT Message (#PCDATA)>
<!ELEMENT Profile (#PCDATA)>
<!ELEMENT Udh (#PCDATA)>
<!ELEMENT SenderID (#PCDATA)>
<!ATTLIST SenderID
    Type (Alpha | Numeric | Shortcode) "Numeric"
>
<!ELEMENT ExpireDate (#PCDATA)>
<!ELEMENT Subscriber (SubscriberNumber?, SessionId?)>
<!ELEMENT SubscriberNumber (#PCDATA)>
<!ELEMENT Operator (#PCDATA)>
<!ELEMENT Tariff (#PCDATA)>
<!ELEMENT SessionId (#PCDATA)>
<!ELEMENT Tag (#PCDATA)>
<!ATTLIST Tag
    Name CDATA #REQUIRED
>
<!ELEMENT Tags (Tag+)>
<!ELEMENT ServiceDesc (#PCDATA)>
<!ELEMENT ContentType (#PCDATA)>
<!ELEMENT ServiceId (#PCDATA)>
```


© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

9.4.4. Explanation of XML nodes in the Notification Request

This section provides a list of all of the defined nodes within the Notification Request, and provides the format and description of each. Element names are written in plain typeface, as <tags>, and attribute names are written in ***italics***. Nodes marked with * are optional within the XML format. However, some of these optional nodes are **required** in certain circumstances for successful submission/routing of an MT message. In each case, the description given for the node makes this clear.

<NotificationRequest> (element)	Format – N/A.
Root element of the Notification Request. Attributes: *Version*.	
Version (attribute)	Format – alphanumeric string.
The version number of the Notification Request (**9.2 Version control**). The current version number is "3.5". It is recommended that the Client always uses the most recent version of the Notification Request, to receive the full benefit from the XML interface and its feature set.	
<NotificationHeader> (element)	Format – N/A.
Contains header information elements.	
<PartnerName> (element)	Format – alphanumeric string.
Account User Name assigned by mBlox (**6.2. Account specific information provided to the Client**).	
<PartnerPassword> (element)	Format – alphanumeric string.
Account Password assigned by mBlox (**6.2. Account specific information provided to the Client**).	
<Username> (element – optional)	Format – alphanumeric string.

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

This element is optional and is provisioned by mBlox should the client wish to have delivery receipts sent to a specific pre-provisioned URL that is different from the default one provisioned on the account.	
<SubscriptionName>* (element – optional)	Format – alphanumeric string (uppercase "A" to "Z", lowercase "a" to "z" and digits "0" to "9" only). Max length: 9 chars.
This element is currently not used by mBlox. The Client may use it to identify MT messages grouped under a single <SubscriptionName>. For example, this may be useful when searching XML documents in log files.	
<NotificationList> (element)	Format – N/A.
Contains a single <Notification> element. Attributes: *BatchID*	
BatchID* (attribute – optional)	Format – numeric string. Max value: 99,999,999
Should be set to unique value which is returned in the Notification Service, in order to identify the Notification Request to which it relates. Refer to **9.4.6 Use of BatchID and SequenceNumber** for further information on the use of BatchID and SequenceNumber.	
<Notification> (element)	Format – N/A.
Contains the data associated with a single MT message. Attributes: *MessageType*, *Format*, *SequenceNumber*.	
SequenceNumber (attribute)	Format – numeric string. Max value: 99 999
Should be set to "1", as only one <Notification> should be submitted in a <NotificationList>. Refer to **9.4.6 Use of BatchID and SequenceNumber** for further information on the use of BatchID and SequenceNumber.	
MessageType* (attribute – optional)	Format – alphanumeric string. Valid values: "SMS", "FlashSMS".

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

Specifies whether message is a standard or flash SMS. Set to "SMS", or omit to send standard message. The Client should refer to the relevant product documentation for information on the support of flash SMS.	
Format* (attribute – optional)	Format – alphanumeric string. Valid values: "Binary", "Unicode", "Imode"
Specifies the format of the <Message> contents as binary, Unicode or Imode. If omitted, the <Message> contents are interpreted as plain text (using a subset of the Latin-1 character set). **10 Message Body Character Sets and Encoding** provides important information on the various supported character sets and encodings. The support of each format is product-specific, and the Client should refer to the relevant product documentation to determine which their supported formats are.	
<Message> (element)	Format – alphanumeric string. Max length (for each *Format*): 160 chars (text), 140 bytes (Binary), 70 chars (Unicode), 333 chars (Imode)
Body of message, encoded as specified by the *Format* attribute. **10 Message Body Character Sets and Encoding** provides information on the various supported character sets and encodings.	
<Profile> (element)	Format – numeric string. Max length: 5 chars.
The ProfileID for the MT product being accessed on the mBlox platform. The Client is provided with valid <Profile> values for their account when it is provisioned (**section 5.2**). Clients who have been provided with a number of ProfileIDs should take particular care to use the correct one when submitting a Notification Request.	
<Udh>* (element – optional)	Format – alphanumeric string. Byte list of format ":a1:a2…"
Contains the optional User Data Header. The Client should refer to the relevant product documentation for more information on the use of the UDH. Note that if a UDH is included it decreases the maximum payload of the message body.	
<SenderID>*	Format – dependent on value of *Type* attribute.

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

(element – optional)	"Alpha" – Alphanumeric string. Max length: 11 chars. "Numeric" – Numeric string. Max length: 15 chars. Includes country code e.g. "447900000000". "Shortcode" – Numeric string. Max length: 15 chars.

Identifier which appears on the subscriber's handset as the originator of the message, in the format specified by the ***Type*** attribute. The support of each format is product-specific, and the Client should refer to the relevant product documentation to determine which they may use. Attributes: ***Type***.

If the format specified, via the ***Type*** attribute, is not supported by the <Profile> used, the submission will fail, and an error code will be returned in the Notification Request Result.

Numeric – displayed as an international MSISDN, i.e. preceded by a "+" symbol.
Shortcode – displayed as a network short code, i.e. without a preceding "+".
Alphanumeric – recommended that it does not contain spaces and is limited to the following characters: lower case "a" to "z", upper case "A" to "Z", digits "0" to "9". Subscribers are unable to reply to "Alpha" originators.

Type* (attribute – optional)	Format – alphanumeric string. Valid values: "Alpha", "Numeric", "Shortcode".

Specifies the format of the <SenderID> contents as alphanumeric, numeric, or a shortcode. If the Type attribute is omitted, the default format is numeric.

<ExpireDate>* (element – optional)	Format – numeric string. UTC date and time (no daylight saving adjustment) in format "MMDDHHmm".

Time and date at which the message will be discarded within the SMSC, if it has not yet been successfully delivered to the subscriber. The maximum time allowed between message submission and expiry is product-specific. The Client should refer to the relevant product documentation. Messages with an <ExpireDate> outside the limits allowed by the <Profile> will have the <ExpireDate> corrected by the mBlox platform, before it is forwarded.

<Operator>* (element – optional)	Format – numeric string. Valid values supplied by mBlox.

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

A numeric identifier (termed the OperatorID), uniquely identifying the operator to which the message should be routed. A valid set of OperatorIDs is provided to the Client by mBlox, and is included in the relevant product documentation. It is principally used for PSMS routing, and may be a required element. Details on its use are provided in the PSMS Implementation Guide for the appropriate geography. **It is important to use the correct <Operator> for the specified subscriber when submitting a Notification Request, otherwise delivery and/or premium billing may fail.**	
<Tariff>* (element – optional)	Format – alphanumeric string. Valid values supplied by mBlox. The tariff for a billed message (applicable to PSMS only, and may be required). If included in a Notification Request with a non-premium ProfileID, it will be ignored. The Client should refer to the PSMS Implementation Guidelines for their geography for information on the format and use of this parameter.
<Subscriber> (element)	Format – N/A. Contains elements related to the individual recipient subscriber. Only one <Subscriber> element should be included in a Notification Request.
<SubscriberNumber> (element)	Format – numeric string. Min length: 6 chars. Max length: 20 chars. Includes country code, e.g. "447900000000". The recipient subscriber's phone number. In some PSMS implementations the phone number must be replaced by a numeric alias, to protect the identity of the subscriber. The Client should refer to the PSMS Implementation Guidelines for their geography for more information. Although this element is optional within the DTD it should always be included.
<SessionId>* (element – optional)	Format – Alphanumeric string. Max length: 45 chars. Indicates the SessionID for the MT message (applicable to PSMS only, and may be **required**). The use of SessionID is geography- and operator-dependent, and the Client should refer to the PSMS Implementation Guidelines in their specific geography for more information.
<Tags>*	Format – N/A.

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

(element – optional) Contains between one and ten <Tag> elements.	
<Tag> (element) <Tag> elements are multi-purpose elements used to carry any parameters that are **required** only for a specific product. The Client should refer to the relevant product documentation to determine whether any <Tag> elements should be submitted in the Notification Request and the required format for these. Individual <Tag> elements are identified by their ***Name***. Attributes: ***Name***.	Format – Varies depending on product.
Name (attribute) Contains an identifier for an individual <Tag> element, i.e. it is the name of the parameter being passed in the contents of the element. The Client should refer to the relevant product documentation to determine the Name of any <Tag> elements that are required.	Format – Alphanumeric string. Valid values dependent on product.
<ServiceDesc>* (element – optional) Applicable to certain PSMS products only. The Client should refer to the relevant product documentation to determine whether this element is required. If present, it contains an identifier which sets a brief service description visible to the recipient subscriber (e.g. on their mobile phone bill). Service descriptions and their related identifiers are provisioned by mBlox when the service is set up.	Format – alphanumeric string ("a" to "z" and "0" to "9" only, not case sensitive). Valid values provisioned by mBlox. Max length: 10 chars.
<ContentType>* (element – optional) Applicable to certain PSMS products only. The Client should refer to the relevant product documentation to determine whether this element is required. If present, it contains a numeric identifier which specifies the type of content being delivered via the MT message. Valid values are provided in the document "mBlox Content Type Guide".	Format – numeric string.
<ServiceID>*	Format – numeric string.

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

(element – optional) Valid values supplied by mBlox.

Applicable to certain PSMS products only. The Client should refer to the relevant product documentation to determine whether this element is required. If present, it contains the ServiceID for the premium service being provided to the recipient subscriber. The Client is provided with valid <ServiceID> values when each new service is set up on their account.

Clients who have been provided with a number of ServiceIDs should take particular care to use the correct one when submitting a Notification Request.

9.4.5. Order of elements

A requirement for a valid XML document is that its elements appear in the order specified in the DTD. The mBlox XML server carries out validation of received XML documents against the DTD.

If a Notification Request is submitted with elements in the wrong order, the Notification Request Result will return indicating a format error in the XML document.

9.4.6. Use of BatchID and SequenceNumber

Although the format of the XML supports multiple <Notification> elements within a <NotificationList>, the Client's application should only send one message in a single Notification Request, i.e. there should only be a single <Notification> with SequenceNumber="1". It is strongly recommended that the BatchID attribute is included in the XML, and used by the Client as a unique identifier for each submitted message. To achieve this, it is suggested that the Client's application should automatically increment the BatchID by 1 for each successive Notification Request.

Notification Services may be enabled on an account (section 5.1.2). If so, each Notification Service XML document will include the same value of BatchID as the Notification Request to which it relates. Clients following these recommendations can, therefore, use BatchID to link related Notification Requests and Notification Services.

9.5. The Notification Request Result

Figure 4 outlines the communication flow involved in acknowledging the receipt of a Notification Request. For simplicity, only one XML server and core platform element are shown, and the IP load balancer (US platform only) is omitted. The mBlox server receives a Notification Request from the Client and forwards the message which it contains to the

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

core platform. The server responds by returning a Notification Request Result within the same HTTP transaction.

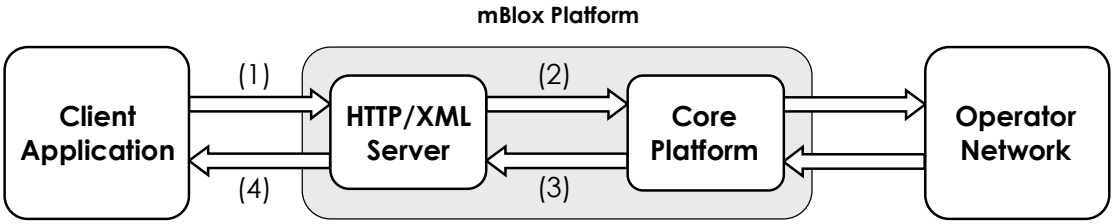

Figure 4 – Data flow in a Notification Request/Notification Request Result transaction

There are two different acknowledgement schemes – Immediate Return (non-blocking mode) or Reported Return (blocking mode). The Client's account settings held by mBlox (**6.1.2 Client preference account settings**) determine which acknowledgement scheme is used.

9.5.1. Immediate Return (non-blocking mode)

In this scheme, the mBlox HTTP server responds to the Notification Request as soon as it has forwarded the message to the mBlox core platform and prior to acceptance/rejection by the core platform. Referring to
Figure 4, event (4) occurs prior to event (3). A Notification Request Result returned as OK at this stage only indicates the following.

- The HTTP transaction has been successfully completed.
- The XML document has been validated and parsed by the mBlox application.
- A correct account User Name/Password combination has been submitted.

The message may still be rejected by the core platform due to other errors (e.g. use of an incorrect profile in the Notification Request), the Notification Service sent to the client indicates the code and reason for the failure, further details are available in 9.6 The Notification Service.

9.5.2. Reported Return (blocking mode)

In this scheme, the mBlox HTTP server only responds to the Notification Request once the core platform has accepted/rejected the message and communicated this back to the server. Referring to **figure 4**, event (4) only occurs after events (2) and (3). A Notification Request Result returned as OK at this stage indicates that the core platform has accepted the message.

9.5.3. Choice between Immediate and Reported Return

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

The default acknowledgement scheme setting for an account is Reported Return (**6.1.2 Client preference account settings**). Reported Return has the advantage of providing full confidence that the mBlox core platform has accepted the message and is capable of routing it to the operator network. Its disadvantage is that the Client's application has to wait longer for the HTTP response to a Notification Request and that the associated TCP socket is blocked during this period. The application must continue to deal with other HTTP transactions while waiting.

If a client is using Immediate Return and the transaction is rejected at step 3 in Figure 4, the client will receive a Notification Service indicating the reason the transaction could not be accepted.

9.5.4. Levels of result code

The Notification Request Result contains up to three hierarchical result codes related to a Notification Request. The three codes are found in the following elements:

- **<RequestResultCode>** - contains a result related to the XML format or header parameters of the Notification Request.
- **<NotificationResultCode>** - contains a result related to whether the parameters in the <Notification> element have the correct syntax.
- **<SubscriberResultCode>** - contains a result related to the whether the individual MT message has been accepted by the mBlox platform.

The lower levels of code only occur if the higher levels have not indicated an error, for example the <SubscriberResultCode> element is only present in the XML document if the <NotificationResultCode> corresponds to "OK".

A retry parameter indicates whether the Client's application should attempt to retry a Notification Request which has returned an error. <RequestResultCode> and <NotificationResultCode> errors should not typically occur if the interface is functioning correctly and will never be accompanied by an indication that the Notification Request should be retried. Large numbers of these errors indicate that the Client's application should be debugged.

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

9.5.5. Format of the Notification Request Result

The Notification Request Result XML document consists of a header, containing account identification data and a result indicator which relates to the Notification Request's XML format and header. This is followed by a list containing message-related and subscriber-related result indicators. Although the format of the XML supports multiple result indicators in this list, the Client's application should only submit a single message to an individual subscriber in a Notification Request and only receive a single set of result indicators in the Notification Request Result.

```
<?xml version="1.0" encoding="ISO-8859-1" ?>
<NotificationRequestResult Version="3.5">
    <NotificationResultHeader>
        <PartnerName>UserName</PartnerName>
        <PartnerRef></PartnerRef>
        <RequestResultCode>0</RequestResultCode>
        <RequestResultText>OK</RequestResultText>
    </NotificationResultHeader>
    <NotificationResultList>
        <NotificationResult SequenceNumber="1">
            <NotificationResultCode>0</NotificationResultCode>
            <NotificationResultText>OK</NotificationResultText>
            <SubscriberResult>
                <SubscriberNumber>447900000000</SubscriberNumber>
                <SubscriberResultCode>0</SubscriberResultCode>
                <SubscriberResultText>OK</SubscriberResultText>
                <Retry>0</Retry>
            </SubscriberResult>
        </NotificationResult>
    </NotificationResultList>
</NotificationRequestResult>
```


© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

9.5.6. Formal XML DTD of the Notification Request Result

```
<!ELEMENT NotificationRequestResult (NotificationResultHeader,
NotificationResultList?)>
<!ATTLIST NotificationRequestResult
    Version CDATA "3.5"
>
<!ELEMENT NotificationResultHeader (PartnerName, PartnerRef,
RequestResultCode, RequestResultText?)>
<!ELEMENT PartnerName (#PCDATA)>
<!ELEMENT PartnerRef (#PCDATA)>
<!ELEMENT RequestResultCode (#PCDATA)>
<!ELEMENT RequestResultText (#PCDATA)>
<!ELEMENT NotificationResultList (NotificationResult+)>
<!ELEMENT NotificationResult (NotificationResultCode, NotificationResultText?,
SubscriberResult*)>
<!ATTLIST NotificationResult
    SequenceNumber CDATA #REQUIRED
>
<!ELEMENT NotificationResultCode (#PCDATA)>
<!ELEMENT NotificationResultText (#PCDATA)>
<!ELEMENT SubscriberResult (SubscriberNumber?, SubscriberResultCode,
SubscriberResultText?, Retry, Operator?)>
<!ELEMENT SubscriberNumber (#PCDATA)>
<!ELEMENT SubscriberResultCode (#PCDATA)>
<!ELEMENT SubscriberResultText (#PCDATA)>
<!ELEMENT Retry (#PCDATA)>
<!ELEMENT Operator (#PCDATA)>
```


© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

9.5.7. Explanation of XML nodes in the Notification Request Result

This section provides a list of all of the defined nodes within the Notification Request Result and provides the format and description of each. Element names are written in plain typeface, as <tags>, and attribute names are written in italics. Nodes marked with * are optional within the XML format. Child elements will not be present if their optional parent element is not present.

<NotificationRequestResult> (element) Root element of the Notification Request Result. Attributes: *Version*.	Format – N/A.
Version (attribute) The version number of the Notification Request Result (**9.2 Version control**). "3.5" is currently in use.	Format – alphanumeric string.
<NotificationResultHeader> (element) Contains header information elements and a result indicator related to the XML format and <NotificationHeader> of the Notification Request.	Format – N/A.
<PartnerName> (element) Corresponds to the <PartnerName> supplied in the Notification Request (**9.4.4 Explanation of XML nodes in the Notification Request**).	Format – alphanumeric string.
<PartnerRef> (element) Not used and is always empty.	Format – empty.
<RequestResultCode> (element)	Format – numeric string.

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

A numeric code value which returns a result related to the XML format and <NotificationHeader> of the Notification Request. The result code may be interpreted as follows. "0" = OK. "3" = Incorrect <PartnerName> or <PartnerPassword>. "9" = Unspecified error.	
<RequestResultText>* (element - optional)	Format – alphanumeric string. Max length: none. If present, provides an explanation of the result returned in <RequestResultCode>.
<NotificationResultList>* (element – optional)	Format – N/A. This element and its child elements will only be included if the value returned in <RequestResultCode> is "0", i.e. the formatting and header information of the Notification Request have been reported as OK. Contains a single <NotificationResult> element.
<NotificationResult> (element)	Format – N/A. Contains the result information associated with the <Notification> element within the Notification Request. Attributes: *SequenceNumber.*
SequenceNumber (attribute)	Format – numeric string. Max value: 99 999 Corresponds to the SequenceNumber value of the <Notification> provided in the Notification Request (**9.4.4 Explanation of XML nodes in the Notification Request**).
<NotificationResultCode> (element)	Format – numeric string.

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

A numeric code value which returns a result related to the <Notification> in the Notification Request. The result code may be interpreted as follows. "0" = OK. "2" = Invalid *Sequence* or no <Message>. "5" = Malformed <SenderID>. "7" = Malformed <Message>. "10" = Malformed <ExpireDate>. "11" = Malformed <BatchID>. "12" = Malformed/unrecognized <Profile>. "13" = Malformed <Udh>. "14" = Malformed <Operator>. "15" = Malformed <Tariff>. "16" = Malformed <ServiceDesc>. "17" = Malformed <ContentType>. "18" = Malformed <ServiceId>.	
<NotificationResultText>* (element – optional)	Format – alphanumeric string. Max length: none. If present, provides an explanation of the result returned in <NotificationResultCode>.
<SubscriberResult>* (element – optional)	Format – N/A. This element and its child elements will only be included if the value returned in <NotificationResultCode> is "0", i.e. the contents of the <Notification> have been correctly formed, and no other error has occurred at the <Notification> level. Contains the individual, subscriber-related result indicator.
<SubscriberNumber> (element)	Format – numeric string. Corresponds to the individual <SubscriberNumber> provided in the Notification Request (**9.4.4 Explanation of XML nodes in the Notification Request**). Although this element is optional within the DTD, it will always be included.
<SubscriberResultCode> (element)	Format – numeric string.

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

A numeric code value which returns the subscriber-related result. If this is returned as "0", the Notification Request Result acknowledges successful submission of the MT message to the mBlox platform. Note that if the acknowledgement scheme is set to Immediate Return, this code may return as "0", even if the message subsequently fails in the mBlox core platform (**9.5.1 Immediate Return**). The result code may be interpreted as follows. "0" = OK. "1" = <Profile> could not be used. "2" = Message could not be forwarded to mBlox core platform. "4" = Client reference could not be found. "5" = Error in message data causing translation error. "6" = Message contains illegal characters causing encoding error. "7" = General sending error. "8" = Private connection configuration error. "9" = Send timed out – success unknown. "10" = Message rejected by mBlox core platform – error code returned in <SubscriberResultText>. "11" = Timeout waiting for mBlox core platform to respond – success of submission to the core platform is unknown. Repeated receipt of <SubscriberResultCode> 11 in response to MT submissions typically indicates that there is a problem in the interaction between the Client's application and the mBlox XML server. The Client should contact mBlox Client Support in this case. "12" = Invalid <Subscriber>. "16" = Invalid <SessionId>. "17" = Both <SubscriberNumber> and <SessionId> missing. "100" = Destination busy.	
<SubscriberResultText>* (element – optional)	Format – alphanumeric string. Max length: none. If present, provides an explanation of the result returned in <SubscriberResultCode>.
<Retry> (element)	Format – numeric string. Valid values: "0", "1".

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

A code which specifies whether the submission of this message should be tried again, and may be interpreted as follows: "0" = Submission should not be tried again. "1" = Submission should be tried again. Refer to **7.3 Retry mechanism** for information regarding retry management.
<Operator>* Format – numeric string. (element – optional) Valid values supplied by mBlox. If present, contains a numeric identifier (termed the OperatorID), uniquely identifying the operator to which the message has been submitted. This element will not be present if the Immediate Return acknowledgement scheme is used. Details on its use are provided in the PSMS Implementation Guide for the appropriate geography.

9.6. The Notification Service

The Client may or may not be configured to receive Notification Services on their account (6.1.2 Client preference account settings).

Notification Services are generated for MT messages on a per-message basis, providing information about the status of the message in the operator network. The status which is returned falls into one of two categories.

- **Intermediate status** – The message is in storage within the operator network and delivery is being attempted
- **Final status** – Indicates the final status of the message. No further status will be provided

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

Table 3 lists the possible states, which are returned in the <Status> element of the XML document (9.6.3

Explanation of XML nodes in the Notification Service).

Status	Type	Interpretation
acked	Intermediate	The operator has accepted the message.
buffered phone	Intermediate	The operator has been unable to deliver the message due to a phone-related issue, but is retrying the message.
buffered smsc	Intermediate	The operator has been unable to deliver the message to an issue in the operator network, but is retrying the message.
Delivered	Final	The message has been successfully delivered to the handset. Note that this does not necessarily indicate that the handset can correctly interpret the data which it has accepted.
Non Delivered	Final	The message has failed somewhere in the operator network.
Lost Notification	Final	Unknown final status, as no delivery receipt has been received by the mBlox platform. The delay before mBlox issues an unknown final status is product-specific.

Table 3 – Possible states returned in Notification Service

A numeric reason code is also returned in the Notification Service. The range of valid codes depends on whether the Premium Transaction Tracking (PTT) product is enabled on the Client account (**6.1.1 Product-dependent account settings**). The Client should refer to the PTT Technical Guide for their geography for interpretation of the reason code.

In normal operation every message which has been accepted by the mBlox core platform will generate a final status Notification Service. This may occur up to 48 hours after the receipt of the original Notification Request. In addition, one or more intermediate status Notification Services may be generated in the interim.

9.6.1. Format of the Notification Service

The Notification Service XML document consists of a header, providing account identification/authentication, followed by a list containing the delivery status information elements. Although the XML format supports multiple <Notification> and <Subscriber>

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

elements in a single Notification Service only one of each element will occur. Hence, each Notification Service provides delivery status information related to a single MT message sent to an individual subscriber.


```
<?xml version="1.0" encoding="ISO-8859-1"?>
<NotificationService Version="2.3">
    <Header>
        <Partner>UserName</Partner>
        <Password>Password</Password>
        <ServiceID>1</ServiceID>
            </Header>
            <NotificationList>
        <Notification BatchID="1" SequenceNumber="1">
            <Subscriber>
                <SubscriberNumber>447900000000</SubscriberNumber>
                <Status>Delivered</Status>
                <TimeStamp>200412181548</TimeStamp>
                <MsgReference>111040222919362</MsgReference>
                <Reason>4</Reason>
                    <Tags>
                      <Tag Name="Tag-RejectCode">13</Tag>
                      <Tag Name="Tag-RejectReason">You cannot send using this
profile.</Tag>
                       <Tag
                    Name="TagSessionID">CGlOzkboqyMAAAQAARTQpkyxN3d8v6PB7P4:3d
                    </Tag>
            </Tags>
        </Subscriber>
                    </Notification>
    </NotificationList>
     </NotificationService>
```


© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

9.6.2. Formal XML DTD of the Notification Service

```
<!ELEMENT NotificationService (Header, NotificationList)>
<!ATTLIST NotificationService
    Version CDATA "2.3"
>
<!ELEMENT Header (Partner, Password, ServiceID)>
<!ELEMENT Partner (#PCDATA)>
<!ELEMENT Password (#PCDATA)>
<!ELEMENT ServiceID (#PCDATA)>
<!ELEMENT NotificationList (Notification+)>
<!ELEMENT Notification (Subscriber+)>
<!ATTLIST Notification
    BatchID CDATA #REQUIRED
    SequenceNumber CDATA #REQUIRED
>
<!ELEMENT Subscriber (SubscriberNumber?, Status, TimeStamp, MsgReference,
Reason? , Tags?)>
<!ELEMENT SubscriberNumber (#PCDATA)>
<!ELEMENT Status (#PCDATA)>
<!ELEMENT TimeStamp (#PCDATA)>
<!ELEMENT MsgReference (#PCDATA)>
<!ELEMENT Reason (#PCDATA)>
<!ELEMENT Tag (#PCDATA)>
<!ATTLIST Tag
      Name CDATA #REQUIRED
>
<!ELEMENT Tags (Tag+)>
```

9.6.3. Explanation of XML nodes in the Notification Service

This section provides a list of all of the defined nodes within the Notification Request Result and provides the format and description of each. Element names are written in plain typeface, as <tags>, and attribute names are written in italics.

<NotificationService> (element)	Format – N/A.
Root element of the Notification Service. Attributes: *Version*.	
Version (attribute)	Format – alphanumeric string.
The version number of the Notification Service (**9.2 Version control**). "2.3" is currently in use.	

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

<Header> (element) Contains header information elements.	Format – N/A.
<Partner> (element) Account User Name assigned by mBlox (**6.2. Account specific information provided to the Client**).	Format – alphanumeric string.
<Password> (element) Account Password assigned by mBlox (**6.2. Account specific information provided to the Client**).	Format – alphanumeric string.
<ServiceID> (element) Always contains the value "1".	Format – alphanumeric string.
<NotificationList> (element) Contains a single <Notification> element.	Format – N/A.
<Notification> (element) Contains a single <Subscriber> element. Attributes: *BatchID*, *SequenceNumber*.	Format – N/A.
BatchID (attribute) Corresponds to the *BatchID* value of the <NotificationList> provided in the Notification Request (**9.4.4 Explanation of XML nodes in the Notification Request**). Identifies the MT message to which the delivery status information relates.	Format – numeric string. Max value: 99 999 999

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

SequenceNumber (attribute)	Format – numeric string. Max value: 99 999
Corresponds to the *SequenceNumber* value of the <Notification> provided in the Notification Request (**9.4.4 Explanation of XML nodes in the Notification Request**).	
<Subscriber> (element)	Format – N/A.
Contains the delivery status information related to a single MT message sent to an individual subscriber.	
<SubscriberNumber> (element)	Format – numeric string. As per <SubscriberNumber> format in Section 9.4.4).
Corresponds to the <SubscriberNumber> provided in the Notification Request (**section 8.4.4**). Although this element is optional within the DTD, it will always be included.	
<Status> (element)	Format – alphanumeric string.
Indicates the delivery status of the MT message. The values "acked", "buffered phone", "buffered smsc", "Delivered", "Non Delivered" & "Lost Notification" may be interpreted as shown in, **9.6 The Notification Service**.	
<TimeStamp> (element)	Format – numeric string. UTC date and time (no daylight saving adjustment) in format "YYYYMMDDHHmm".
Time and date at which the delivery receipt was received by the mBlox platform.	
<MsgReference> (element)	Format – alphanumeric string.
An internal mBlox reference for the message to which the Notification Service relates.	
<Reason> (element - optional)	Format – numeric string.

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

If present, a numeric code indicating the reason for the MT message delivery status. The basic codes correspond to the value returned in the <Status> element, and may be interpreted as follows.

"1" = buffered phone
"2" = buffered smsc
"3" = acked
"4" = Delivered
"5" = Non Delivered
"6" = Lost Notification

The Premium Transaction Tracking product (available in some countries) extends the set of reason codes beyond the basic codes listed above. The Client should refer to the Premium Transaction Tracking Technical Guide for their geography for more information.

If the client account is set to immediate return and the message is subsequently rejected by the mBlox core platform, there will be two additional Reason codes returned, as follows.
"100" = The message has been rejected, please DO NOT retry the message.
"101" = The message has been rejected, please retry the message immediately

These reject reason codes will be accompanied by two additional tags - Tag-RejectCode and Tag-RejectReason, possible values of the Reject Code and Reason are below:

<Tags>* (element – optional)	Format – N/A
Contains between one and ten <Tag> elements	
<Tag> (element)	Format – Varies depending on product
<Tag> elements are multi-purpose elements used to carry any parameters that are provided only for a specific product. The Client should refer to the relevant product	

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

documentation to determine whether any <Tag> elements should be expected in the Response Service and the format for these. Individual <Tag> elements are identified by their Name. Attributes: Name.

9.7. The Response Service

The provision of MO messages via the Response Service is product-specific (section 5.1.1).

9.7.1. Format of the Response Service

The Response Service XML document consists of a header, providing account identification/authentication, followed by a list of the messages being submitted to the Client application. Although the format of the XML supports multiple MO messages in this list, the mBlox platform will only submit one message per XML, i.e. there will only ever be one <Response> element. The <Response> element contains various items of message-specific data and the message body.

```
<?xml version="1.0" encoding="ISO-8859-1" standalone="yes"?>
<ResponseService Version="2.3">
    <Header>
        <Partner>UserName</Partner>
        <Password>Password</Password>
        <ServiceID>1</ServiceID>
    </Header>
    <ResponseList>
        <Response SequenceNumber="1" Type="SMS" Format="Text">
            <TransactionID>215980</TransactionID>
            <OriginatingNumber>447900000000</OriginatingNumber>
            <Time>200412181427</Time>
            <Data>This is an MO message.</Data>
            <Deliverer>1</Deliverer>
            <Destination>12345</Destination>
            <Operator>99999</Operator>
            <Tariff>150</Tariff>
            <SessionId>SessionID</SessionId>
            <Tags>
                <Tag Name="Number">12</Tag>
                <Tag Name="City">Rome</Tag>
            </Tags>
        </Response>
    </ResponseList>
</ResponseService>
```


© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

9.7.2. Formal XML DTD of the Response Service

```
<!ELEMENT ResponseService (Header, ResponseList)>
<!ATTLIST ResponseService
    Version CDATA "2.3"
>
<!ELEMENT Header (Partner, Password, ServiceID)>
<!ELEMENT Partner (#PCDATA)>
<!ELEMENT Password (#PCDATA)>
<!ELEMENT ServiceID (#PCDATA)>
<!ELEMENT ResponseList (Response+)>
<!ELEMENT Response (TransactionID, OriginatingNumber?, Time, Data, Deliverer,
Destination, Operator?, Tariff?, Udh?, MsgReference?, SessionId?, Tags?)>
<!ATTLIST Response
    SequenceNumber CDATA #REQUIRED
    Type CDATA "SMS"
    Format (Text | Unicode | Binary | Imode) #IMPLIED
>
<!ELEMENT TransactionID (#PCDATA)>
<!ELEMENT OriginatingNumber (#PCDATA)>
<!ELEMENT Time (#PCDATA)>
<!ELEMENT Data (#PCDATA)>
<!ELEMENT Deliverer (#PCDATA)>
<!ELEMENT Destination (#PCDATA)>
<!ELEMENT Operator (#PCDATA)>
<!ELEMENT Tariff (#PCDATA)>
<!ELEMENT Udh (#PCDATA)>
<!ELEMENT MsgReference (#PCDATA)>
<!ELEMENT SessionId (#PCDATA)>
<!ELEMENT Tag (#PCDATA)>
<!ATTLIST Tag
    Name CDATA #REQUIRED
>
<!ELEMENT Tags (Tag+)>
```

9.7.3. Explanation of XML nodes in the Response Service

This section provides a list of all of the defined nodes within the Response Service, and provides the format and description of each. Element names are written in plain typeface, as <tags>, and attribute names are written in italics. Nodes marked with * are optional within the XML format.

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

<ResponseService> (element) Root element of the Response Service. Attributes: *Version*.	Format – N/A.
Version (attribute) The version number of the Response Service (**9.2 Version control**). "2.3" is currently in use.	Format – alphanumeric string.
<Header> (element) Contains header information elements.	Format – N/A.
<Partner> (element) Account User Name assigned by mBlox (**6.2. Account specific information provided to the Client**).	Format – alphanumeric string.
<Password> (element) Account Password assigned by mBlox (**6.2. Account specific information provided to the Client**).	Format – alphanumeric string.
<ServiceID> (element) Always contains the value "1".	Format – alphanumeric string.
<ResponseList> (element)	Format – N/A.

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

Contains a single <Response> element.	
<Response> (element) Contains the data associated with a single MO message. Attributes: *SequenceNumber, Type, Format*.	Format – N/A.
SequenceNumber (attribute) Always contains the value "1".	Format – numeric string.
Type (attribute) Always contains the value "SMS".	Format – alphanumeric string.
Format* (attribute – optional) Specifies the format of the <Data> contents as plain text, binary, Unicode or Imode. If omitted, the default format is plain text. **10 Message Body Character Sets and Encoding** provides information on the various supported character sets and encodings.	Format – alphanumeric string. Valid values: "Text", "Binary", "Unicode" "Imode".
<TransactionID> (element) Contains an internal mBlox sequence number for the MO message to which the Response Service relates.	Format – numeric string.
<OriginatingNumber> (element)	Format – numeric string. Includes country code, e.g. "447900000000".

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

The originating subscriber's phone number. In some PSMS implementations the phone number is replaced by a numeric alias, to protect the identity of the subscriber. The Client should refer to the Implementation Guidelines for PSMS in their specific geography for more information. Although this element is optional within the DTD, it will always be included. Note that the stated format is only guaranteed for non-premium MO messages. For premium SMS, the phone number is passed transparently by mBlox in the format in which it has been received from the operator network. The exact format is operator-dependent and is not specified by mBlox. The Client should establish the format used by each operator during end-to-end testing.	
<Time> (element)	Format – numeric string. Date and time in format "YYYYMMDDHHmm". Time and date at which the MO message was originated, if supplied by the operator network. In this instance the time will be set to the time zone used by the operator's SMSC. If this is not supplied by the operator network, the UTC time and date at which the message was received by the mBlox platform will be substituted.
<Data> (element)	Format – alphanumeric string. Body of message encoded as specified in the *Format* attribute. Characters will be passed transparently by the mBlox platform in the same encoding as originated by the operator network. **10 Message Body Character Sets and Encoding** provides information on the various supported character sets and encodings.
<Deliverer> (element)	Format – alphanumeric string. Contains a value identifying the operator from which the message has been received. A valid set of values is provided to the Client by mBlox, and is included in the relevant product documentation.
<Destination> (element)	Format – numeric string. Either network short code or MSISDN format, e.g. "447900000000". Contains the destination number to which the MO message has been sent.
<Operator>*	Format – numeric string.

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

(element – optional)	Valid values supplied by mBlox. If present, contains a numeric identifier (termed the OperatorID), uniquely identifying the operator from which the message has been received. Details on its use are provided in the PSMS Implementation Guide for the appropriate geography.
<Tariff>* (element – optional)	Format – alphanumeric string. Only applicable to premium MO messages. If present, contains the tariff for the billed message. The Client should refer to the PSMS Implementation Guidelines for their geography for information on the format and use of this parameter.
<Udh>* (element – optional)	Format – alphanumeric string. Byte list of format ":a1:a2…". If present, contains the User Data Header. The Client should refer to the relevant product documentation for more information on the use of the UDH.
<MsgReference>* (element-optional)	Format – alphanumeric string. If present, contains a reference (provided by the operator network) for the MO message to which the Response Service relates.
<SessionId>* (element – optional)	Format – Alphanumeric string. Max length: 45 chars. Indicates the SessionID for the MO message (applicable to PSMS only). The use of SessionID is geography and operator-dependent and the Client should refer to the PSMS Implementation Guidelines in their specific geography for more information.
<Tags>* (element – optional)	Format – N/A. Contains between one and ten <Tag> elements.
<Tag>	Format – Varies depending on product.

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

(element) <Tag> elements are multi-purpose elements used to carry any parameters that are provided only for a specific product. The Client should refer to the relevant product documentation to determine whether any <Tag> elements should be expected in the Response Service and the format for these. Individual <Tag> elements are identified by their *Name*. Attributes: *Name*.	
Name (attribute)	Format – Alphanumeric string. Valid values dependent on product.
Contains an identifier for an individual <Tag> element, i.e. it is the name of the parameter being passed in the contents of the element. The Client should refer to the relevant product documentation to determine the *Name* of any <Tag> elements that are expected.	

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

10. Message Body Character Sets and Encoding

10.1. Illegal characters in XML

Certain characters are illegal within the contents of XML elements, as they cause confusion when the XML is parsed. These are ", ', <, >, and &. There are two methods which may be used by the Client to transmit these characters safely.

10.1.1. Entity references

An illegal character in XML may be replaced by an entity reference, as shown in **Table 4**.

Illegal character		Replace with
<	Less than	<
>	Greater than	>
&	Ampersand	&
'	Apostrophe	'
"	Quotation mark	"

Table 4 - XML entity references

This method will be used by the mBlox platform to encode these illegal characters when they occur in the body of MO messages being submitted to the Client. A similar method is used to encode Unicode characters in MT messages (**10.2.2 Unicode**).

10.1.2. CDATA

This method may be used to transmit a message body with a large number of illegal characters, in a more efficient way than replacing each one with an entity reference. The entire message is placed in a CDATA section, which begins with the string "<![CDATA[", and ends with the string "]]>". The contents of the CDATA section are ignored by the XML parser.

```
<Message><![CDATA[This contains illegal chars "<>'&]]></Message>
```


© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

10.2. MT message characters sets and encoding

The Client may choose between different encoding formats for the body of an MT message, depending on the content which is being transmitted. This section relates to the encoding of the message body, not to the encoding of the XML document as a whole.

10.2.1. Plain text

If the format of the message is set to text, the character set supported by the mBlox platform is a subset of ISO-8859-1 (Latin-1), as defined in **Table 5**. The Table shows each character along with its hexadecimal code point (the ISO-8859-1 code point).

00	**01**	**02**	**03**	**04**	**05**	**06**	**07**	**08**	**09**	**0A**	**0B**	**0C**	**0D**	**0E**	**0F**
										LF			CR		
10	**11**	**12**	**13**	**14**	**15**	**16**	**17**	**18**	**19**	**1A**	**1B**	**1C**	**1D**	**1E**	**1F**
20	**21**	**22**	**23**	**24**	**25**	**26**	**27**	**28**	**29**	**2A**	**2B**	**2C**	**2D**	**2E**	**2F**
SP	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
30	**31**	**32**	**33**	**34**	**35**	**36**	**37**	**38**	**39**	**3A**	**3B**	**3C**	**3D**	**3E**	**3F**
0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
40	**41**	**42**	**43**	**44**	**45**	**46**	**47**	**48**	**49**	**4A**	**4B**	**4C**	**4D**	**4E**	**4F**
@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
50	**51**	**52**	**53**	**54**	**55**	**56**	**57**	**58**	**59**	**5A**	**5B**	**5C**	**5D**	**5E**	**5F**
P	Q	R	S	T	U	V	W	X	Y	Z					_
60	**61**	**62**	**63**	**64**	**65**	**66**	**67**	**68**	**69**	**6A**	**6B**	**6C**	**6D**	**6E**	**6F**
	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
70	**71**	**72**	**73**	**74**	**75**	**76**	**77**	**78**	**79**	**7A**	**7B**	**7C**	**7D**	**7E**	**7F**
p	q	r	s	t	u	v	w	x	y	z					
80	**81**	**82**	**83**	**84**	**85**	**86**	**87**	**88**	**89**	**8A**	**8B**	**8C**	**8D**	**8E**	**8F**
90	**91**	**92**	**93**	**94**	**95**	**96**	**97**	**98**	**99**	**9A**	**9B**	**9C**	**9D**	**9E**	**9F**
A0	**A1**	**A2**	**A3**	**A4**	**A5**	**A6**	**A7**	**A8**	**A9**	**AA**	**AB**	**AC**	**AD**	**AE**	**AF**
	¡		£	¤	¥		§								
B0	**B1**	**B2**	**B3**	**B4**	**B5**	**B6**	**B7**	**B8**	**B9**	**BA**	**BB**	**BC**	**BD**	**BE**	**BF**
															ż
C0	**C1**	**C2**	**C3**	**C4**	**C5**	**C6**	**C7**	**C8**	**C9**	**CA**	**CB**	**CC**	**CD**	**CE**	**CF**
				Ä	Å	Æ	Ç		É						
D0	**D1**	**D2**	**D3**	**D4**	**D5**	**D6**	**D7**	**D8**	**D9**	**DA**	**DB**	**DC**	**DD**	**DE**	**DF**
	Ñ					Ö		Ø				Ü			ß
E0	**E1**	**E2**	**E3**	**E4**	**E5**	**E6**	**E7**	**E8**	**E9**	**EA**	**EB**	**EC**	**ED**	**EE**	**EF**
à				ä	å	æ		è	é			ì			
F0	**F1**	**F2**	**F3**	**F4**	**F5**	**F6**	**F7**	**F8**	**F9**	**FA**	**FB**	**FC**	**FD**	**FE**	**FF**
	ñ	ò				ö		ø	ù			ü			

Table 5 – Plain text character set and code values (hexadecimal)

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

If the Client sends a character which is not listed in table 5, the message may fail at any point in the delivery process.

10.2.2. Unicode

The support of Unicode is product-specific. The Client should refer to the relevant product documentation to establish whether they can send in this format.

If the format of the message is set to Unicode, the character set supported by the mBlox platform is Unicode (UCS) and must be encoded as two bytes per character. This is useful for sending characters outside the set defined for the plain text format. As two bytes are required for each character, the message length is limited to 70 Unicode characters.

Unicode characters will be sent transparently to the subscriber and it is up to the receiving device to decode them. In general, any receiving device will only be able to interpret a subset of the full Unicode character set and the Client should be aware of this limitation.

In the Unicode character set, characters with code points up to 0x00FF are identical to those in Latin-1. These may be passed as single characters in the same, straightforward manner as in the plain text format message body. Characters with codes points above 0x00FF must be encoded using the entity reference method, where each Unicode character is encoded in the following format.

```
&#x<two-byte hexadecimal Unicode character code>;
```

For example, the following message body (with Format="Unicode")

```
<Message>An example Unicode character - &#x20AC;</Message>
```

Will be displayed (on a handset capable of interpreting this Unicode character)

10.2.3. Binary

The support of binaries is product-specific. The Client should refer to the relevant product documentation to establish whether they can send in this format.

If the format of the message is set to binary, the characters within the message body are used to encode a string of bytes. Each byte is encoded as a colon followed by two hexadecimal digits, representing the value of the byte, e.g.

```
<Message>:0e:12:c3:03:9f:87:da:e4</Message>
```


© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

The use of the characters "a" to "f" as hexadecimal digits is not case sensitive. This form of encoding is also used for the User Data Header.

The successful delivery of a binary message to a subscriber's handset does not indicate that the device can correctly interpret the format and content of the data received. This is highly dependent on handset manufacturer and model. The Client should ensure that they are sending data which is correctly formatted for the recipient device.

10.2.4. i-mode

Please refer to relevant Product Implementation Guidelines as i-mode is only available in a small number of markets.

10.3. MO message character sets and encoding

The body of an MO message, supplied in the Response Service XML document, may be in any of the formats described in section 9.2. For non-premium, plain text MO messages, the character set is the same as that defined in **Table 5.** However, for premium SMS, characters will be passed transparently from the operator network to the Client's application and mBlox cannot guarantee the character set used. For example, a message body in plain text format may contain Latin-1 characters/code points outside those listed in **Table 5**. The Client should establish the character sets and encoding applicable to each operator during end-to-end testing.

The Client's application should tolerate unexpected characters, and always acknowledge the receipt of a Response Service with an HTTP response (section 7.1).

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

11. Appendix – Example XML Documents for Notification Request

The examples in this Appendix are designed to be useful as a reference only. Note that the correct use of binary format messages and the User Data Header is dependent on the recipient handset. The Client should consult the relevant manufacturer's documentation for information. mBlox does not modify or convert the byte sequence supplied in the Notification Request in any way. It is the responsibility of the Client to ensure that the format and content is correct.

11.1. Plain text SMS

```
<?xml version="1.0"?>
<NotificationRequest Version="3.5">
    <NotificationHeader>
        <PartnerName>MyUserName</PartnerName>
        <PartnerPassword>MyPassword</PartnerPassword>
    </NotificationHeader>
    <NotificationList BatchID="1">
        <Notification SequenceNumber="1">
            <Message>This is a test message.</Message>
            <Profile>9999</Profile>
            <SenderID Type="Shortcode">12345</SenderID>
            <ExpireDate>03212210</ExpireDate>
            <Operator>99999</Operator>
            <Tariff>150</Tariff>
            <Subscriber>
                <SubscriberNumber>447900000000</SubscriberNumber>
            </Subscriber>
        </Notification>
    </NotificationList>
</NotificationRequest>
```

This Notification Request submits a plain text SMS, for delivery to the phone number +447900000000, using the ProfileID 9999. The originator will be displayed in the handset as the network short code "12345" and, if undelivered, the message will expire in the SMSC on 21 March at 10:10 pm. The SMS is a premium MT message to be routed to the operator with the network code 99999, and has an associated tariff of 150.

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

11.2. Ringtone delivered as a binary format SMS


```
<NotificationRequest Version="3.5">
    <NotificationHeader>
        <PartnerName>MyUserName</PartnerName>
        <PartnerPassword>MyPassword</PartnerPassword>
    </NotificationHeader>
    <NotificationList BatchID="1">
        <Notification SequenceNumber="1" MessageType="SMS" Format="Binary">
            <Message>:02:4A:3A:61:4D:A5:B5:C1:CD:BD:B9:CC:04:1E:93:1C:41:0A:30
            :C5:1A:62:8C:31:46:98:A3:0C:51:A8:28:C3:08:41:46:98:A3:6C:59:16:28:DB:
            16:45:8B:28:C5:91:82:8D:B0:84:16:45:8B:2C:C5:96:62:CA:31:65:98:B3:2C:59
            :68:2C:A3:08:41:65:98:B2:30:59:18:2C:8C:16:46:0A:36:C5:91:42:C8:C1:64:6
            0:B2:30:59:16:00:00</Message>
            <Profile>9999</Profile>
            <Udh>:06:05:04:15:81:15:81</Udh>
            <Subscriber>
                <SubscriberNumber>447900000000</SubscriberNumber>
            </Subscriber>
        </Notification>
    </NotificationList>
</NotificationRequest>
```


This Notification Request submits a binary message (including UDH), which will be interpreted as a ringtone, if delivered to a Nokia 6310i.

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

11.3. Operator logo delivered as a binary format SMS

```
<NotificationRequest Version="3.5">
    <NotificationHeader>
        <PartnerName>MyUserName</PartnerName>
        <PartnerPassword>MyPassword</PartnerPassword>
    </NotificationHeader>
    <NotificationList BatchID="1">
        <Notification SequenceNumber="1" MessageType="SMS" Format="Binary">
            <Message>:32:F4:51:00:48:0E:01:00:00:00:00:00:00:00:1E:F0:00:00:00:06:00:
            00:00:12:90:00:00:3F:86:00:00:00:12:90:00:00:3F:C6:00:00:00:1E:F0:00:00:30:
            66:00:00:00:00:00:1B:CF:30:66:1F:18:33:DE:F0:1F:FF:B0:C6:3F:8C:62:52:90:1
            C:F1:BF:86:71:C6:C2:52:90:18:61:BF:C6:60:C3:83:DE:F0:18:61:B0:E6:60:C3:8
            0:00:00:18:61:B0:66:60:C3:83:DE:F0:18:61:B0:66:71:C6:C2:52:90:18:61:BF:C
            6:3F:8C:62:52:90:18:61:BF:86:1F:18:33:DE:F0</Message>
            <Profile>9999</Profile>
            <Udh>:06:05:04:15:82:00:00</Udh>
            <Subscriber>
                <SubscriberNumber>447900000000</SubscriberNumber>
            </Subscriber>
        </Notification>
    </NotificationList>
</NotificationRequest>
```

This Notification Request submits a binary message (including UDH), which will be interpreted as an operator logo, if delivered to a Nokia 6310i on Vodafone UK. Note that the correct operator must be encoded for the logo to be displayed correctly.

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

11.4. WAP push SI delivered as a binary format SMS

```
<NotificationRequest Version="3.5">
    <NotificationHeader>
        <PartnerName>MyUserName</PartnerName>
        <PartnerPassword>MyPassword</PartnerPassword>
    </NotificationHeader>
    <NotificationList BatchID="1">
        <Notification SequenceNumber="1" MessageType="SMS" Format="Binary">
            <Message>:81:06:03:AE:81:EA:03:05:6A:00:45:C6:0C:03:61:62:63:31:2E:6D:
            62:6C:6F:78:00:85:03:6D:62:6C:6F:78:6C:6F:67:6F:74:36:38:2E:67:69:66:00:0
            A:C3:07:20:04:07:16:12:49:15:10:C3:07:20:04:07:23:12:49:15:01:03:20:6D:42
            :6C:6F:78:20:54:36:38:69:20:6C:6F:67:6F:20:00:01:01</Message>
            <Profile>9999</Profile>
            <Udh>:06:05:04:0B:84:23:F0</Udh>
            <SenderID Type="Alpha">Test</SenderID>
            <Subscriber>
                <SubscriberNumber>447900000000</SubscriberNumber>
            </Subscriber>
        </Notification>
    </NotificationList>
</NotificationRequest>
```

This Notification Request submits a binary message (including UDH), which will be interpreted as a WAP push SI, if delivered to a Sony Ericsson T68i. The alert will point to the URL **http://abc1.mblox.com/mbloxlogot68.gif** (this resource does not exist), with the title "mBlox T68i logo".

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

12. Appendix A: Reject Codes and Reasons

RejectCode	RejectReason
3	The prefix of the destination number is blacklisted
6	The mBlox server is currently too busy.
7	You are not allowed to use this value in the **reply** parameter.
8	Syntax error, the parameters are not specified correctly.
10-12	General problem.
13	You cannot send using this **profile**.
14	The username used when you logged in is faulty.
15	You are not allowed to send binary messages using this **profile**.
16-31	There is currently no available route for this message.
32	Error while trying to use a special functionality named "locally forced".
33	Error while trying to use a special functionality called "originator indexing". The originator for this index could not be found.
34	The available destination queues are full.
35	The client account is currently blocked on this server.
36	You supplied an illegal value in the **billingRef** parameter.
37	You are not allowed to send messages to this destination operator.
38	There is currently too much incoming traffic on this destination operator.
39	This sequence has been used earlier among the X last sent messages on this server. X is most likely 50, but it can vary.
42-43	Premium services only. The destination cannot be found for your supplied values. Verify the values used in the **originator**, **tariff** and **operator** fields.
44	There is no available route that supports the requested message features.
45	This product does not support the supplied features. Please verify your use of the **profile** parameter.
46	The text content of this message is prohibited on this product.
47	The number portability operator lookup failed.
48	The operator parameter is required when a MT is sent through the PsmsPlex application.
49	The MT could not be routed in the PsmsPlex application.

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

50	An unknown exception encountered when handling tags.
51	The tag is not configured in the Database.
52	The name of the tag is not valid.
53	The value of the tag is not valid.
54	The tag is not allowed for this destination operator.
55	Syntax error in tag/value pair.
56	Too many tags are submitted in the message.
57	A tag is duplicated.
58	Invalid ServiceDesc. Do not retry.
59	Default ServiceDesc not configured. Do not retry.
60	Invalid ContentType. Do not retry.
61	Default ContentType not configured. Do not retry.
62	ContentType not configured for Operator. Do not retry.
63	Invalid ContentType. Do not retry.
64	Default ServiceId not configured. Do not retry.
65	ServiceId not configured for Operator. Do not retry.

© 2010 mBlox Inc.
Confidential. May not be reproduced or released without the written consent of mBlox.

13. Notes

Please use this space to record any important information or further questions you have on the XML Interface.

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.

Notes (continued)…

© 2010 mBlox Inc.

Confidential. May not be reproduced or released without the written consent of mBlox.